

# EJERCICIOS TEMA 5.

## REALIZACIÓN DE PRUEBAS Y TESTING

**Enlace a mi repositorio de GitHub:**

[https://github.com/byElectro/UD5\\_Testing\\_RafaM](https://github.com/byElectro/UD5_Testing_RafaM)

**Oráculo de pruebas:**

**En todos los ejercicios se utiliza como oráculo la comparación entre el resultado obtenido y el esperado.**

**En los casos de error, se verifica el lanzamiento de excepciones.**

### Ejercicio 1. Método signo

Se proporciona el siguiente método

```
public static int signo(int x) {  
    if (x > 0) {  
        return 1;  
    } else if (x < 0) {  
        return -1;  
    }  
    return 0;  
}
```

#### Tareas

1. Calcula la complejidad ciclomática.
2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
3. Implementa la clase SignoTest con JUnit.

#### Análisis del método

El método signo recibe un número entero y devuelve un valor que indica su signo.

##### Entradas:

- Un número entero x

##### Salidas:

- Devuelve 1 si el número es positivo
- Devuelve -1 si el número es negativo
- Devuelve 0 si el número es cero

##### Casos normales:

- Números positivos y negativos

##### Casos límite:

- Valor 0

**Cobertura de ramas:**

- TC01: rama  $x > 0$
- TC02: rama  $x < 0$
- TC03: rama  $x == 0$

**2. Complejidad ciclomática****Decisiones:**

- if ( $x > 0$ )
- else if ( $x < 0$ )

$2 + 1$  (el que hay de base) = 3

Mínimo tiene que hacerse 3 tests

**3. Casos de prueba**

| ID   | DESCRIPCION  | ENTRADA | ESPERADO | ESTADO |
|------|--------------|---------|----------|--------|
| TC01 | Num positivo | 7       | 1        | OK     |
| TC02 | Num negativo | -3      | -1       | OK     |
| TC03 | Num cero     | 0       | 0        | OK     |

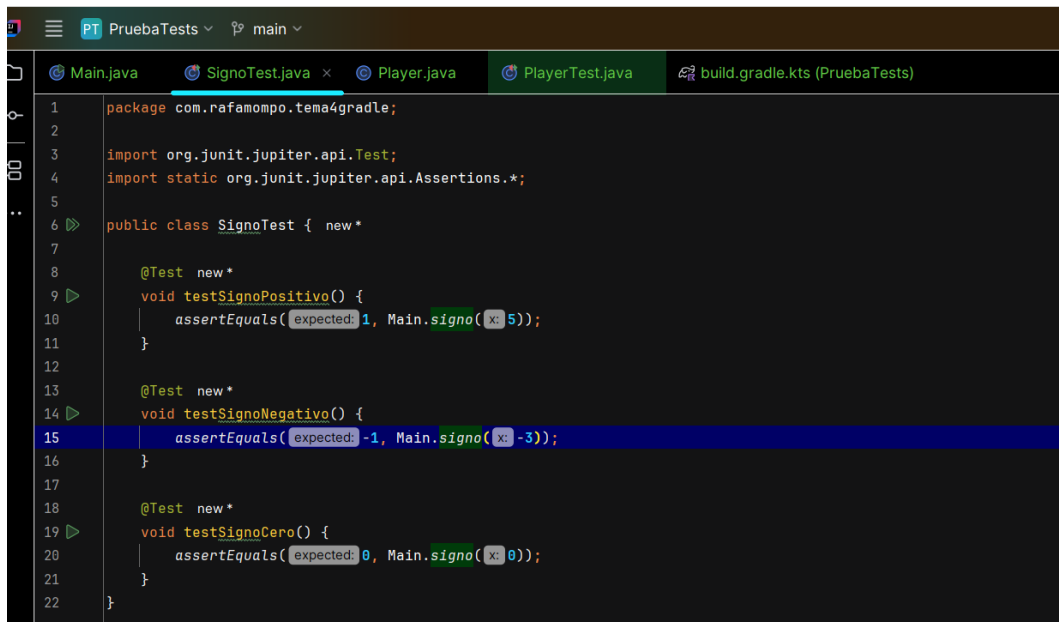
**Oráculo**

Comparación con el resultado esperado.

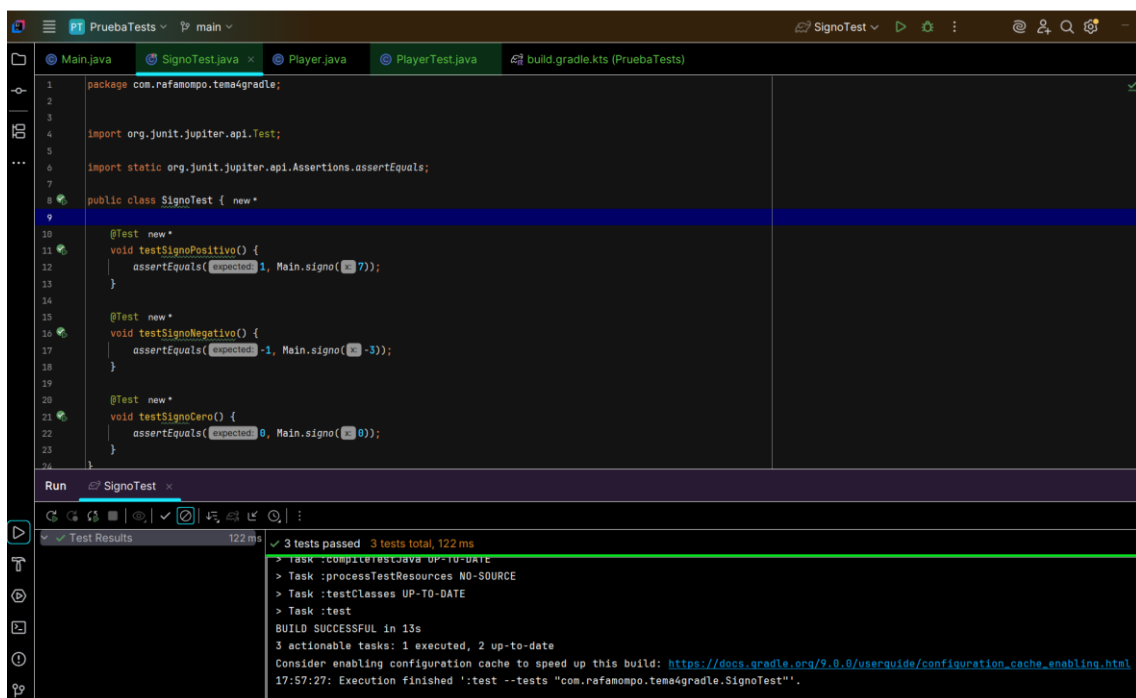
## Captura del método y la clase que he implementado:



```
5 public class Main { new *
32 public static int signo(int x) { no usages new *
33     if (x > 0) {
34         return 1;
35     } else if (x < 0) {
36         return -1;
37     }
38     return 0;
39 }
40 }
```



```
1 package com.rafamompo.tema4gradle;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.*;
5
6 public class SignoTest { new *
7
8     @Test new *
9     void testSignoPositivo() {
10         assertEquals(expected: 1, Main.signo(x: 5));
11     }
12
13     @Test new *
14     void testSignoNegativo() {
15         assertEquals(expected: -1, Main.signo(x: -3));
16     }
17
18     @Test new *
19     void testSignoCero() {
20         assertEquals(expected: 0, Main.signo(x: 0));
21     }
22 }
```



```
1 package com.rafamompo.tema4gradle;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5
6 public class SignoTest { new *
7
8     @Test new *
9     void testSignoPositivo() {
10         assertEquals(expected: 1, Main.signo(x: 7));
11     }
12
13     @Test new *
14     void testSignoNegativo() {
15         assertEquals(expected: -1, Main.signo(x: -3));
16     }
17
18     @Test new *
19     void testSignoCero() {
20         assertEquals(expected: 0, Main.signo(x: 0));
21     }
22 }
```

Run SignoTest x

Test Results 122 ms

3 tests passed 3 tests total 122 ms

> Task :compileTestJava UP-TO-DATE

> Task :processTestResources NO-SOURCE

> Task :testClasses UP-TO-DATE

> Task :test

BUILD SUCCESSFUL in 13s

3 actionable tasks: 1 executed, 2 up-to-date

Consider enabling configuration cache to speed up this build: [https://docs.gradle.org/9.0.0/userguide/configuration\\_cache\\_enabling.html](https://docs.gradle.org/9.0.0/userguide/configuration_cache_enabling.html)

17:57:27: Execution finished ':test --tests "com.rafamompo.tema4gradle.SignoTest"'

**Ejercicio 2. Método clasificarEdad**

Se proporciona el siguiente método:

```
public static String clasificarEdad(int edad) {  
    if (edad < 0) {  
        throw new IllegalArgumentException("Edad no válida");  
    }  
    if (edad < 6) {  
        return "Infancia";  
    } else if (edad < 12) {  
        return "Niñez";  
    } else if (edad < 18) {  
        return "Adolescencia";  
    } else if (edad < 25) {  
        return "Juventud";  
    } else if (edad < 60) {  
        return "Adulthood";  
    } else {  
        return "Vejez";  
    }  
}
```

**Análisis del método**

El método `clasificarEdad` recibe un número entero que representa una edad y devuelve una categoría de edad.

**Entradas:**

- Entero edad

**Salidas:**

- String con la categoría: Infancia, Niñez, Adolescencia, Juventud, Adulthood o Vejez

- Excepción si la edad es negativa

**Casos normales:**

- Edades dentro del rango 0 en adelante

**Casos límite:**

- Edad 0
- Cambios de rango: 6, 12, 18, 25, 60

**Cobertura de ramas:**

- TC01: edad < 0 (excepción)
- TC02: edad < 6
- TC03: edad < 12
- TC04: edad < 18
- TC05: edad < 25
- TC06: edad < 60
- TC07: else (vejez)
- TC08–TC09: valores límite

**Tarea****1. Calcula la complejidad ciclomática.**

Hay 6 decisiones entre los ifs y elifs + 1 base. En total sería  $M = 7$

**2/3.- Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.**

Añade además casos límite especialmente relevantes.

| ID   | DESCRIPCION     | ENTRADA | ESPERADO        | ESTADO |
|------|-----------------|---------|-----------------|--------|
| TC01 | Edad negativa   | -5      | Excepcion       | OK     |
| TC02 | Infancia        | 3       | Infancia        | OK     |
| TC03 | Niñez           | 11      | Niñez           | OK     |
| TC04 | Adolescencia    | 16      | Adolescencia    | OK     |
| T005 | Juventud        | 20      | Juventud        | OK     |
| TC06 | Adulthood       | 35      | Adulthood       | OK     |
| TC07 | Vejez           | 80      | Vejez           | OK     |
| TC08 | Límite Infancia | 5       | Límite Infancia | OK     |
| TC09 | Límite niñez    | 6       | Límite niñez    | OK     |

**Oráculo****Oráculo**

Comparación con el resultado esperado y verificación de excepción en los casos inválidos.

**Implementa ClasificarEdadTest.**

```

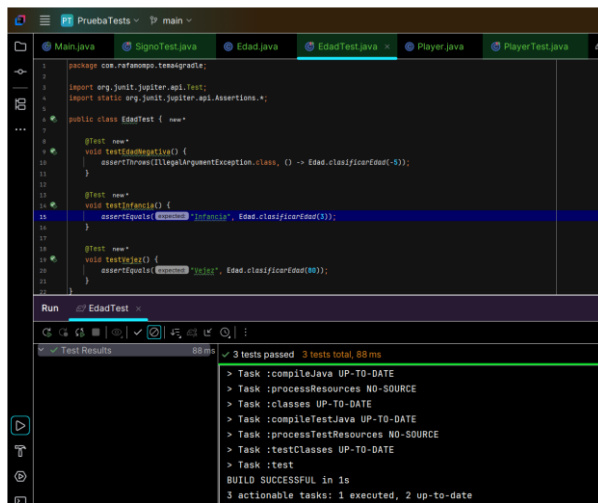
package com.rafaompo.tema4gradle;

public class Edad { no usages new*

    public static String clasificarEdad(int edad) { no usages new*
        if (edad < 0) {
            throw new IllegalArgumentException("Edad no válida");
        }
        if (edad < 6) return "Infancia";
        else if (edad < 12) return "Niñez";
        else if (edad < 18) return "Adolescencia";
        else if (edad < 25) return "Juventud";
        else if (edad < 60) return "Adulthood";
        else return "Vejez";
    }
}

```

## Captura de los test de esta clase:



The screenshot shows an IDE with the following components:

- Editor:** Contains the `EdadTest` class with three test methods:

```
1 package com.rafael.mompó.tenagrade;  
2 import org.junit.jupiter.api.Test;  
3 import static org.junit.jupiter.api.Assertions.*;  
4  
5 public class EdadTest {  
6     @Test  
7     void testEdadNegativa() {  
8         assertThrows(IllegalArgumentException.class, () -> Edad.classificadorEdad(-5));  
9     }  
10  
11     @Test  
12     void testInfancia() {  
13         assertEquals("Infancia", Edad.classificadorEdad(3));  
14     }  
15  
16     @Test  
17     void testJefe() {  
18         assertEquals("Jefe", Edad.classificadorEdad(80));  
19     }  
20 }  
21
```
- Run Console:** Shows the execution of the tests:

```
Run EdadTest  
3 tests passed 3 tests total, 88 ms  
Task :compileJava UP-TO-DATE  
Task :processResources NO-SOURCE  
Task :classes UP-TO-DATE  
Task :compileTestJava UP-TO-DATE  
Task :processTestResources NO-SOURCE  
Task :testClasses UP-TO-DATE  
Task :test  
BUILD SUCCESSFUL in 1s  
3 actionable tasks: 1 executed, 2 up-to-date
```



**Ejercicio 3. Método contarPositivos**

Se proporciona el siguiente método:

```
public static int contarPositivos(int[] datos) {  
    int contador = 0;  
    for (int i = 0; i < datos.length; i++) {  
        if (datos[i] > 0) {  
            contador++;  
        }  
    }  
    return contador;  
}
```

**Análisis del método**

El método contarPositivos recorre un array de enteros y cuenta cuántos valores son mayores que 0.

**Entradas:**

- Array de enteros (int[])

**Salidas:**

- Número entero con la cantidad de valores positivos

**Casos normales:**

- Array con mezcla de valores positivos, negativos y ceros

**Casos límite:**

- Array vacío

- Array sin valores positivos

**Cobertura de ramas:**

- TC01: no entra al bucle (array vacío)
- TC02: entra al bucle pero no se cumple la condición (todos  $\leq 0$ )
- TC03: entra al bucle y se cumple la condición (hay positivos)

**Tareas**

1. Calcula la complejidad ciclomática.

$$M = 2 \text{ decisiones} + 1 = 3$$

2. Diseña pruebas para cubrir:

- o array vacío,
- o array con todos negativos,
- o array con mezcla de positivos, negativos y ceros.

| ID   | DESCRIPCION       | ENTRADA    | ESPERADO | ESTADO |
|------|-------------------|------------|----------|--------|
| TC01 | Array vacío       | {}         | 0        | OK     |
| TC02 | Todos negativos   | {-1,-2,-3} | -1       | OK     |
| TC03 | Mezcla de valores | {-1,0,5,2} | 2        | OK     |

**Oráculo**

Comparación con el resultado esperado.

3. Justifica qué ramas cubre cada prueba.

La complejidad ciclomática del método es 3, por lo que existen tres caminos independientes en el programa.

Los casos de prueba diseñados cubren las siguientes ramas:

- **Caso 1:**  
No entra al bucle → array vacío
- **Caso 2:**  
Entra en el bucle pero nunca se cumple la condición → todos los valores son negativos o cero
- **Caso 3:**  
Entra en el bucle y se cumple la condición → existen valores positivos

#### 4. Implementa ContarPositivosTest.

The screenshot shows an IDE with the following components:

- Editor:** Displays the code for `ContadorPositivosTest`. The code includes imports for JUnit and Assertions, and three test methods: `testArrayVacio`, `testTodosNegativos`, and `testMezcla`. The `testMezcla` method is currently selected and highlighted.
- Run Panel:** Shows the execution of `ContadorPositivosTest`. It includes a toolbar with icons for running, debugging, and other actions.
- Test Results:** A summary bar indicating "3 tests passed 3 tests total, 30 ms".
- Output Console:** Displays the build process logs, showing tasks like `compileJava`, `processResources`, `classes`, `compileTestJava`, `processTestResources`, `testClasses`, and `test`, concluding with "BUILD SUCCESSFUL in 1s".

```
1 package com.rafaompo.tema4gradle;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.*;
5
6 public class ContadorPositivosTest { new *
7
8     @Test new *
9     void testArrayVacio() {
10         assertEquals(expected: 0, Contador.contarPositivos(new int[]{}));
11     }
12
13     @Test new *
14     void testTodosNegativos() {
15         assertEquals(expected: 0, Contador.contarPositivos(new int[]{-1, -2, -3}));
16     }
17
18     @Test new *
19     void testMezcla() {
20         assertEquals(expected: 2, Contador.contarPositivos(new int[]{-1, 0, 5, 2}));
21     }
22 }
```

Run ContadorPositivosTest ×

✓ Test Results 30 ms ✓ 3 tests passed 3 tests total, 30 ms

> Task :compileJava  
> Task :processResources NO-SOURCE  
> Task :classes  
> Task :compileTestJava  
> Task :processTestResources NO-SOURCE  
> Task :testClasses  
> Task :test  
BUILD SUCCESSFUL in 1s

**Ejercicio 4. Método calificacion**

Se proporciona el siguiente método:

```
public static String calificacion(int nota) {  
    if (nota < 0 || nota > 10) {  
        throw new IllegalArgumentException("Nota no válida");  
    }  
    return switch (nota) {  
        case 0, 1, 2, 3, 4 -> "SUSPENSO";  
        case 5 -> "SUFICIENTE";  
        case 6 -> "BIEN";  
        case 7, 8 -> "NOTABLE";  
        default -> "SOBRESALIENTE";  
    };  
}
```

**Análisis del método**

El método `calificacion` recibe una nota numérica y devuelve una calificación en texto.

**Entradas:**

- Entero nota (0 a 10)

**Salidas:**

- String con la calificación:

SUSPENSO, SUFICIENTE, BIEN, NOTABLE o SOBRESALIENTE

- Excepción si la nota no es válida

**Casos normales:**

- Notas entre 0 y 10

**Casos límite:**

- Valores frontera: 0, 4, 5, 6, 7, 8, 10

- Valores inválidos: <0 o >10

**Cobertura de ramas:**

- TC01: validación (nota inválida)
- TC02: case 0–4
- TC03: case 5
- TC04: case 6
- TC05: case 7–8
- TC06: default (9–10)

**Tareas****1. Calcula la complejidad ciclomática**

$$M = 6$$

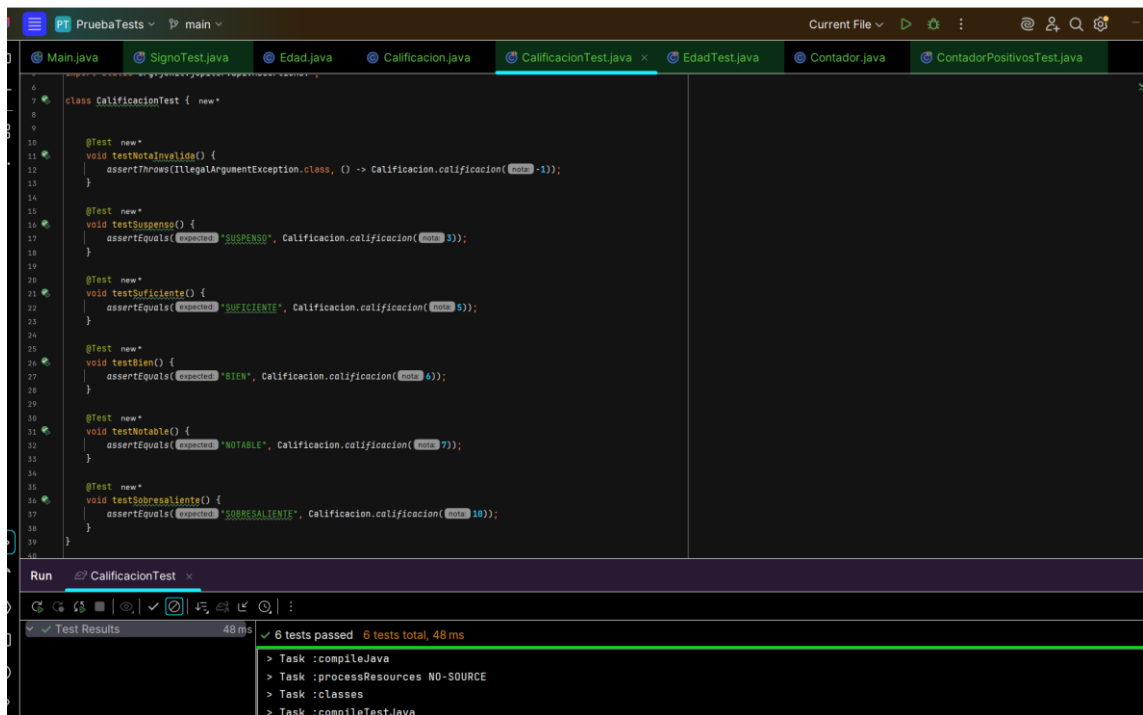
**2. Diseña los casos mínimos para cubrir todas las alternativas del switch y la validación inicial.**

| ID   | DESCRIPCION   | ENTRADA | ESPERADO      | ESTADO |
|------|---------------|---------|---------------|--------|
| TC01 | Nota inválida | -1      | Excepcion     | OK     |
| TC02 | Suspenso      | 3       | SUSPENSO      | OK     |
| TC03 | Suficiente    | 5       | SUFICIENTE    | OK     |
| TC04 | Bien          | 6       | BIEN          | OK     |
| T005 | Notable       | 8       | NOTABLE       | OK     |
| TC06 | Sobresaliente | 10      | SOBRESALIENTE | OK     |

**Oráculo**

**Comparación con el resultado esperado y verificación de excepción en los casos inválidos.**

### 3. Implementa CalificacionTest.



```
1 class CalificacionTest {  
2     new  
3  
4  
5  
6  
7     @Test new  
8     void testNotaInvalida() {  
9         | assertThrows(IllegalArgumentException.class, () -> Calificacion.calificacion(1));  
10    }  
11  
12  
13  
14  
15     @Test new  
16     void testSuspense() {  
17         | assertEquals(expected "SUSPENSO", Calificacion.calificacion(3));  
18    }  
19  
20  
21  
22     @Test new  
23     void testSuficiente() {  
24         | assertEquals(expected "SUFICIENTE", Calificacion.calificacion(5));  
25    }  
26  
27  
28  
29     @Test new  
30     void testBien() {  
31         | assertEquals(expected "BIEN", Calificacion.calificacion(6));  
32    }  
33  
34  
35  
36     @Test new  
37     void testNotable() {  
38         | assertEquals(expected "NOTABLE", Calificacion.calificacion(7));  
39    }  
40  
41  
42  
43     @Test new  
44     void testSobresaliente() {  
45         | assertEquals(expected "SOBRESALIENTE", Calificacion.calificacion(10));  
46    }  
47 }  
48  
49  
50
```

Run CalificacionTest

Test Results 48 ms ✓ 6 tests passed 6 tests total, 48 ms

- > Task :compileJava
- > Task :processResources NO-SOURCE
- > Task :classes
- > Task :compileTestJava

### 4. Indica qué casos podrían agruparse con @ParameterizedTest.

Los casos que pueden agruparse mediante @ParameterizedTest son aquellos que, teniendo diferentes valores de entrada, producen el mismo resultado esperado.

En este caso, se pueden agrupar:

- Las notas entre 0 y 4, que devuelven "SUSPENSO".
- Las notas 7 y 8, que devuelven "NOTABLE".
- Las notas 9 y 10, que devuelven "SOBRESALIENTE".

No se agrupan los casos de 5 (SUFICIENTE) ni 6 (BIEN), debido a que ya representan resultados únicos.

### **Ejercicio 5. Crear un método esBisiesto**

**Implementa un método con la siguiente firma:**

**public static boolean esBisiesto(int anyo)**

**Reglas**

**Un año es bisiesto si:**

- **es divisible por 4 y no por 100, o**
- **es divisible por 400.**

#### **Análisis del método**

**El método esBisiesto determina si un año es bisiesto según reglas del calendario.**

**Entradas:**

- **Entero anyo**

**Salidas:**

- **Boolean (true o false)**

**Casos normales:**

- **Años positivos**

**Casos límite:**

- **Años divisibles por 4, 100 y 400**
- **Años no divisibles por 4**



**Cobertura de ramas:**

- TC01: divisible por 4 y no por 100
- TC02: divisible por 100 y no por 400
- TC03: divisible por 400
- TC04: no divisible por 4

**Tareas****1. Implementa el método.**

```
public static boolean esBisiesto(int anyo) {  
    return (anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 ==  
0);  
}
```

**2. Calcula su complejidad ciclomática a partir de tu código.**

La complejidad ciclomática es 4, porque hay 3 condiciones dentro del return.

Esto hace que existan 4 casos posibles:

- Es divisible por 4 y no por 100
- Es divisible por 100 pero no por 400
- Es divisible por 400
- No es divisible por 4

**3. Diseña los casos de prueba necesarios para cubrir todas las decisiones.**

| ID   | DESCRIPCION        | ENTRADA | ESPERADO | ESTADO |
|------|--------------------|---------|----------|--------|
| TC01 | Divisible por 4    | 2024    | True     | OK     |
| TC02 | Divisible por 100  | 1900    | False    | OK     |
| TC03 | Divisible por 400  | 2000    | True     | OK     |
| TC04 | No divisible por 4 | 2023    | False    | OK     |

**Oráculo**

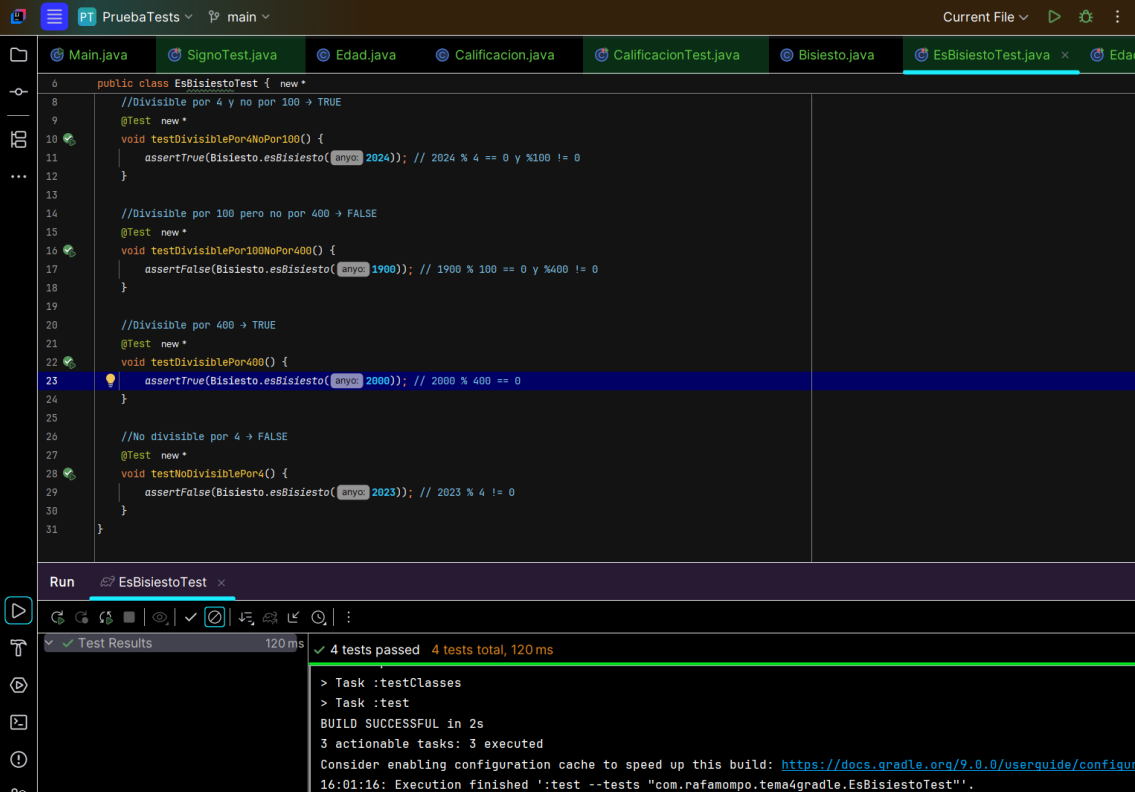
**Comparación con el resultado esperado.**

**4. Implementa EsBisiestoTest.**

**Requisitos**

**Debes incluir al menos estos tipos de caso:**

- divisible por 4 y no por 100,
- divisible por 100 y no por 400,
- divisible por 400,
- no divisible por 4.



The screenshot shows an IDE with the following components:

- File Explorer:** Shows a project named "PruebaTests" with a "main" folder. The "main" folder contains several files: Main.java, SignoTest.java, Edad.java, Calificacion.java, CalificacionTest.java, Bisiesto.java, EsBisiestoTest.java, and Edad.java.
- Editor:** Displays the code for `EsBisiestoTest.java`. The code is as follows:

```
6 public class EsBisiestoTest { new *
8     //Divisible por 4 y no por 100 → TRUE
9     @Test new *
10    void testDivisiblePor4NoPor100() {
11        | assertTrue(Bisiesto.esBisiesto(2024)); // 2024 % 4 == 0 y %100 != 0
12    }
13
14    //Divisible por 100 pero no por 400 → FALSE
15    @Test new *
16    void testDivisiblePor100NoPor400() {
17        | assertFalse(Bisiesto.esBisiesto(1900)); // 1900 % 100 == 0 y %400 != 0
18    }
19
20    //Divisible por 400 → TRUE
21    @Test new *
22    void testDivisiblePor400() {
23        | assertTrue(Bisiesto.esBisiesto(2000)); // 2000 % 400 == 0
24    }
25
26    //No divisible por 4 → FALSE
27    @Test new *
28    void testNoDivisiblePor4() {
29        | assertFalse(Bisiesto.esBisiesto(2023)); // 2023 % 4 != 0
30    }
31 }
```
- Run Panel:** Shows the execution results for the `EsBisiestoTest` class. The results are as follows:

```
Run EsBisiestoTest
Test Results 120 ms
4 tests passed 4 tests total, 120 ms
> Task :testClasses
> Task :test
BUILD SUCCESSFUL in 2s
3 actionable tasks: 3 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.0.0/userguide/configu
16:01:16: Execution finished 'test --tests "com.refamompo.tema4gradle.EsBisiestoTest"'
```

## Ejercicio 6. Implementar Piedra – Papel – Tijera

Implementa el método:

```
public static String jugar(String jugador1, String jugador2)
```

Las jugadas posibles son:

- "PIEDRA"
- "PAPEL"
- "TIJERA"

Reglas

- Piedra gana a tijera
- Tijera gana a papel
- Papel gana a piedra
- Si ambas jugadas son iguales --> "EMPATE"
- Si alguna jugada no es válida --> `IllegalArgumentException`

Cobertura de ramas:

- TC01: empate (`jugador1 == jugador2`)
- TC02–TC04: gana jugador1
- TC05: gana jugador2
- TC06–TC07: validación de entrada (excepción)

## Análisis del método

El método jugar simula el juego Piedra-Papel-Tijera entre dos jugadores.

### Entradas:

- Dos Strings: jugador1 y jugador2

### Salidas:

- "JUGADOR1", "JUGADOR2" o "EMPATE"
- Excepción si alguna jugada no es válida

### Casos normales:

- Todas las combinaciones válidas del juego

### Casos límite:

- Entradas inválidas
- Jugadas iguales (empate)

## Tareas

### 1. Implementar el método.

```
2. package com.rafamompo.tema4gradle;  
  
public class Juego {  
  
    public static String jugar(String jugador1, String jugador2)  
    {  
  
        // Validación de entradas  
        if (!jugador1.equals("PIEDRA") &&  
            !jugador1.equals("PAPEL") && !jugador1.equals("TIJERA") ||  
            !jugador2.equals("PIEDRA") &&  
            !jugador2.equals("PAPEL") && !jugador2.equals("TIJERA")) {  
            throw new IllegalArgumentException("Jugada no  
válida");  
        }  
    }  
}
```

```

        // Empate
        if (jugador1.equals(jugador2)) {
            return "EMPATE";
        }

        // Reglas del juego
        if (jugador1.equals("PIEDRA") &&
            jugador2.equals("TIJERA")) return "JUGADOR1";
        if (jugador1.equals("TIJERA") &&
            jugador2.equals("PAPEL")) return "JUGADOR1";
        if (jugador1.equals("PAPEL") &&
            jugador2.equals("PIEDRA")) return "JUGADOR1";

        return "JUGADOR2";
    }
}

```

### 3. Calcular la complejidad ciclomática.

La complejidad ciclomática del método es 7.

Esto se debe a que el código contiene 6 decisiones:

- Validación de entradas (jugadas válidas)
- Comparación de empate
- 3 condiciones para que gane el jugador 1

Según la fórmula:

$M = \text{número de decisiones} + 1$

$M = 6 + 1 = 7$

### 4. Diseñar los tests mínimos para cubrir todas las ramas.

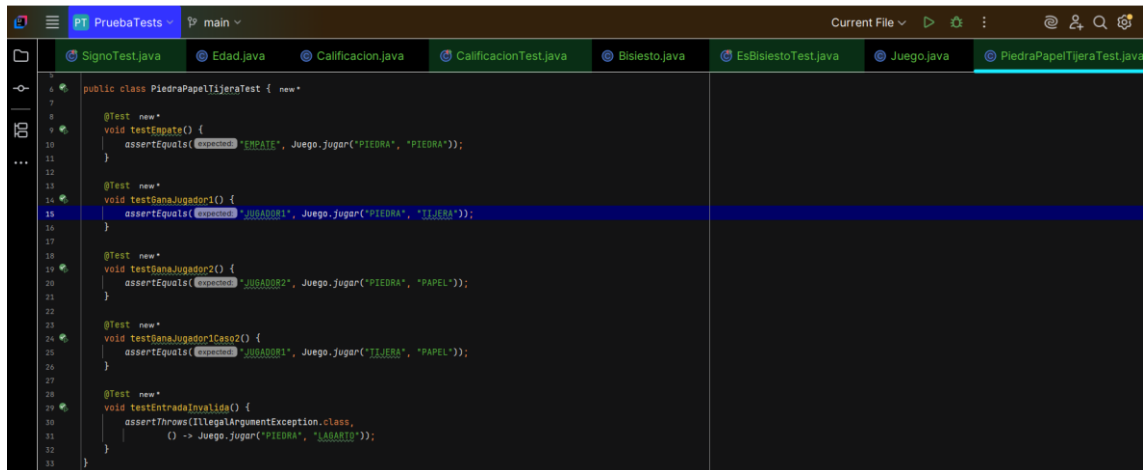
| ID   | Descripción         | Entrada            | Esperado  | Estado |
|------|---------------------|--------------------|-----------|--------|
| TC01 | Empate              | PIEDRA,<br>PIEDRA  | EMPATE    | OK     |
| TC02 | Gana J1             | PIEDRA,<br>TIJERA  | JUGADOR1  | OK     |
| TC03 | Gana J1             | TIJERA,<br>PAPEL   | JUGADOR1  | OK     |
| TC04 | Gana J1             | PAPEL,<br>PIEDRA   | JUGADOR1  | OK     |
| TC05 | Gana J2             | PIEDRA,<br>PAPEL   | JUGADOR2  | OK     |
| TC06 | Entrada<br>inválida | PIEDRA,<br>LAGARTO | Excepción | OK     |

|      |                  |                  |           |    |
|------|------------------|------------------|-----------|----|
| TC07 | Entrada inválida | INVALIDO, TIJERA | Excepción | OK |
|------|------------------|------------------|-----------|----|

## Oráculo

Comparación con el resultado esperado y verificación de excepción en los casos inválidos.

### 5. Implementar PiedraPapelTijeraTest.



```
public class PiedraPapelTijeraTest {  
    @Test  
    void testEmpate() {  
        assertEquals("EMPATE", Juego.jugar("PIEDRA", "PIEDRA"));  
    }  
  
    @Test  
    void testGanaJugador1() {  
        assertEquals("JUGADOR1", Juego.jugar("PIEDRA", "TIJERA"));  
    }  
  
    @Test  
    void testGanaJugador2() {  
        assertEquals("JUGADOR2", Juego.jugar("PIEDRA", "PAPEL"));  
    }  
  
    @Test  
    void testGanaJugadorCaso2() {  
        assertEquals("JUGADOR1", Juego.jugar("TIJERA", "PAPEL"));  
    }  
  
    @Test  
    void testEntradaInvalida() {  
        assertThrows(IllegalArgumentException.class,  
            () -> Juego.jugar("PIEDRA", "¡¡¡¡¡"));  
    }  
}
```

### **Ejercicio 7. Implementar Validador de contraseñas**

**Implementa el método:**

**public static int evaluarPassword(String password)**

**Reglas**

**Se asignan puntos:**

- **+1 si tiene al menos 8 caracteres**
- **+1 si tiene mayúsculas**
- **+1 si tiene minúsculas**
- **+1 si tiene números**
- **+1 si tiene símbolos**

**Interpretación del resultado:**

- **entre 0 y 2 --> "DEBIL"**
- **entre 3 y 4 --> "MEDIA"**
- **5 --> "FUERTE"**

**Cobertura de ramas**

- **TC01: puntos <= 2 (DEBIL)**
- **TC02: puntos 3–4 (MEDIA)**
- **TC03: puntos = 5 (FUERTE)**
- **TC04: combinación numérica**
- **TC05: sin símbolos**



### **Análisis del método**

**El método evaluarPassword analiza la seguridad de una contraseña y la clasifica según su fortaleza.**

#### **Entradas:**

- String password

#### **Salidas:**

- "DEBIL", "MEDIA" o "FUERTE"

#### **Casos normales:**

- Contraseñas con diferentes combinaciones de letras, números y símbolos

#### **Casos límite:**

- Contraseñas vacías o muy cortas
- Contraseñas con solo un tipo de carácter

**Tareas****1. Implementar el método.**

```
public class ValidadorContraseñas {  
    public static String evaluarPassword(String password) {  
        int puntos = 0;  
  
        if (password.length() >= 8) puntos++;  
  
        if (password.matches(".*[A-Z].*")) puntos++;  
  
        if (password.matches(".*[a-z].*")) puntos++;  
  
        if (password.matches(".*\\d.*")) puntos++;  
  
        if (password.matches(".*[^A-Za-z0-9].*")) puntos++;  
  
        if (puntos <= 2) return "DEBIL";  
        if (puntos <= 4) return "MEDIA";  
        return "FUERTE";  
    }  
}
```

**2. Calcular complejidad ciclométrica.**

$$M = 7 + 1 = 8$$

### 3. Diseñar casos de prueba representativos.

| ID   | Descripción  | Entrada     | Esperado | Estado |
|------|--------------|-------------|----------|--------|
| TC01 | Muy débil    | "abc"       | DEBIL    | OK     |
| TC02 | Media        | "abc12345"  | MEDIA    | OK     |
| TC03 | Fuerte       | "Abc123\$%" | FUERTE   | OK     |
| TC04 | Solo números | "12345678"  | MEDIA    | OK     |
| TC05 | Sin símbolos | "Abcdef12"  | MEDIA    | OK     |

### Oráculo

Comparación con el resultado esperado.

### 4. Implementar PasswordTest.

The screenshot shows an IDE with the following components:

- Editor:** Displays the code for `PasswordTest.java`. It includes imports for JUnit 5 and assertions, and four test methods: `testDebil()`, `testMedia()`, `testFuerte()`, and `testSoloNumeros()`. Each method uses `assertEquals()` to compare the expected result with the output of `ValidadorContraseñas.evaluarPassword()`.
- Run Panel:** Shows the execution of `PasswordTest`. It indicates that 4 tests passed out of 4 tests total, taking 49 ms.
- Task List:** A list of tasks executed during the build process, including `compileJava UP-TO-DATE`, `processResources NO-SOURCE`, `classes UP-TO-DATE`, `compileTestJava`, `processTestResources NO-SOURCE`, `testClasses`, and `test`.
- Build Status:** The build is marked as `BUILD SUCCESSFUL in 1s`.
- Actionable Tasks:** A summary showing 3 actionable tasks, with 2 executed and 1 up-to-date.
- Path:** The bottom status bar shows the file path: `PruebaTests > src > test > java > com > rafamompo > tema4gradle > PasswordTest > testSoloNumeros`.